DALHOUSIE UNIVERSITY

Manipulating and Visualizing Data

Tables, joins, and graphics in R and Python

Prof. Gabriel Spadon, PhD

Faculty of Computer Science

spadon@dal.ca

1 Where this class sits

What it is for This class moves between Data Understanding and Data Preparation.

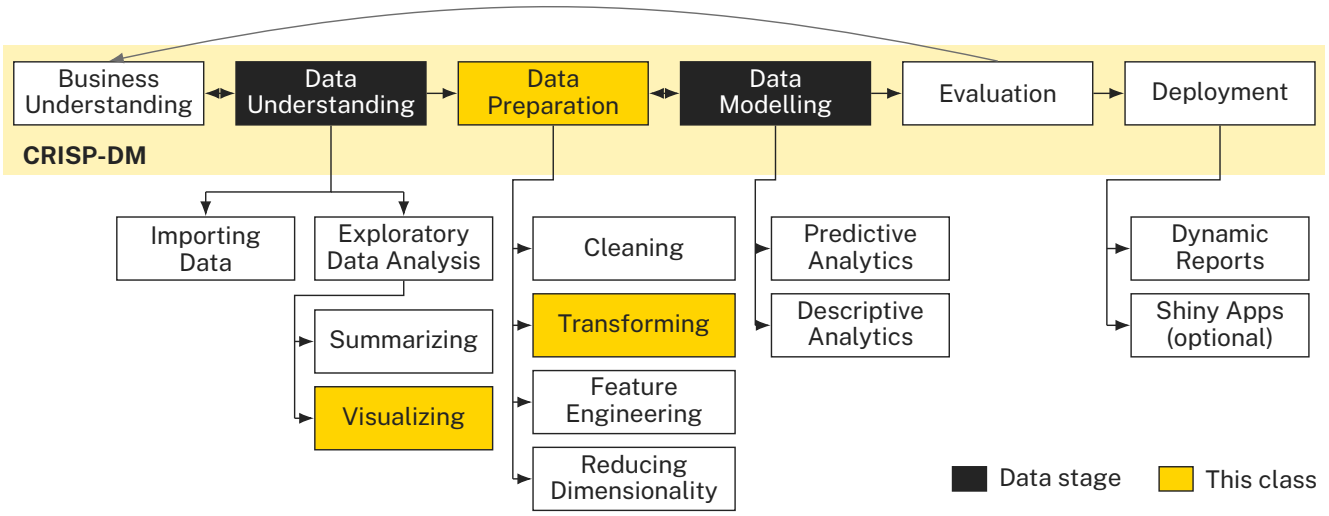


Figure 1. Manipulation and visualization connect preparation with exploration.

The starting point A table operation changes the available evidence. Before any operation, say what one row represents; in tidy data each observation forms a row [9].

2 Table objects and setup

What it is for The same bracket expression returns different objects in different table classes. Knowing which one you hold avoids silent errors when a function expects a table and receives a vector.

The rules A tibble is a subclass of `data.frame` with its own printing and subsetting methods. It prints a compact preview with column types.

- In a base data frame, `df[, "x"]` simplifies to a vector.
- In a tibble, `tb[, "x"]` stays a one-column table. Use `tb[["x"]]` to extract the vector.
- In pandas, `tb[["x"]]` is a DataFrame (a table) and `tb["x"]` is a Series (one labelled column).

Both base data frames and tibbles keep strings as character values by default. Tibbles do not partially match column names. Convert with `as.data.frame` when a function requires base data-frame behaviour, and check a function's documented input when moving between table classes.

Setup Use R 4.1 or newer for the native pipe, `dplyr` 1.1.1 or newer for the join-relationship check, and `ggplot2`. Python uses NumPy, pandas, Matplotlib, seaborn and scikit-learn. The scikit-learn Iris loader supplies the Iris measurements without a network download. Install missing packages in your course environment, and run the code in a working folder where output files can be saved.

R

```r
library(dplyr)
library(ggplot2)
```

Python

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
```

Worked example The R block prints the classes of a whole tibble, of one base-frame column (`integer`), of `tb[, "x"]` (still a tibble) and of `tb[["x"]]` (`integer`). The Python block prints `DataFrame` and `Series`.

R

```r
tb <- tibble(x = 1:3, y = c("a", "b", "c"))
df <- as.data.frame(tb)
print(class(tb))
print(class(df[, "x"]))
print(class(tb[, "x"]))
print(class(tb[["x"]]))
```

Python

```python
tb = pd.DataFrame({"x": [1, 2, 3],
                   "y": ["a", "b", "c"]})
print(type(tb[["x"]]))  # DataFrame, a table
print(type(tb["x"]))  # Series, labelled column
```

3 The game data

What it is for Every calculation in the manipulation sections uses the same two small tables, so each result can be checked by hand.

The tables `plays` has eight records. One record is one player–game pair, identified by `player` and `game` together. `hours` is time played in hours and `achieved` counts the achievements earned. `games` is a lookup with one row per game, giving a `genre` and an achievement `total`.

Three deliberate cases These drive the later checks.

- Dan's Portal hours are unknown.
- Tunic has a play record but no lookup row.
- Stardew has a lookup row but no play record.

Keep `plays` and `games` unchanged. Every example assigns its result to a new name.

R

```r
# Player-game records.
plays <- tibble(
  player = c("ana", "ana", "ben", "ben",
             "cara", "cara", "dan", "dan"),
  game = c("Portal", "Celeste", "Portal",
           "Hades", "Celeste", "Hades",
           "Portal", "Tunic"),
  hours = c(12.5, 30, 8, 45.5, 22, 17.5, NA, 6),
  achieved = c(9L, 14L, 4L, 25L, 11L, 8L,
               2L, 3L))
```

Python

```python
# Player-game records.
plays = pd.DataFrame({
    "player": ["ana", "ana", "ben", "ben",
               "cara", "cara", "dan", "dan"],
    "game": ["Portal", "Celeste", "Portal",
             "Hades", "Celeste", "Hades",
             "Portal", "Tunic"],
    "hours": [12.5, 30, 8, 45.5, 22, 17.5,
              np.nan, 6],
    "achieved": [9, 14, 4, 25, 11, 8, 2, 3]})
```

R

```r
# Game lookup.
games <- tibble(
  game = c("Portal", "Celeste", "Hades",
           "Stardew"),
  genre = c("puzzle", "platformer",
            "roguelike", "farming"),
  total = c(15L, 30L, 49L, 40L))
print(plays)
print(games)
```

Python

```python
# Game lookup.
games = pd.DataFrame({
    "game": ["Portal", "Celeste", "Hades",
             "Stardew"],
    "genre": ["puzzle", "platformer",
              "roguelike", "farming"],
    "total": [15, 30, 49, 40]})
print(plays)
print(games)
```

4 The six operations

What they are for Each operation answers one question about the table. Which records? Which variables? In what order? What new quantity? Within which groups? What summary?

The operations `filter` keeps rows, `select` keeps columns, `arrange` reorders rows, `mutate` adds a derived column, `group_by` declares groups and `summarise` collapses rows to one per group [10]. Grouping by itself does not collapse records; the later summary decides the output rows. Figure 2 shows how each operation changes a table.

`filter`	Keep rows that meet a condition
`select`	Keep or reorder columns
`arrange`	Reorder rows
`mutate`	Add, replace, or remove columns
`group_by`	Define groups without removing rows
`summarise`	Return a summary row per group

Figure 2. Each operation has a distinct effect on a table.

5 Select records

What it is for Keep only the records that answer the question.

The rule `filter` keeps the rows whose condition is `TRUE`. Comma-separated conditions are combined with AND. Pandas uses a Boolean mask with `loc`, a `query` expression, or membership through `isin`.

Worked example `hours > 20` keeps three records: Ana–Celeste (30), Ben–Hades (45.5) and Cara–Celeste (22). Adding `player != "ana"` leaves Ben–Hades and Cara–Celeste.

Pandas keeps the input index after filtering. The printed labels `1, 3, 4` are the positions of the retained rows in the original default index, not an error. Keep a meaningful record identifier in a column, and reset an index only when the operation needs a new positional index.

R

```r
print(filter(plays, hours > 20))
print(filter(plays, hours > 20,
             player != "ana"))
print(filter(plays,
             game %in% c("Hades", "Tunic")))
```

Python

```python
print(plays.loc[plays["hours"] > 20])
print(plays.query(
    "hours > 20 and player != 'ana'"))
print(plays.loc[plays["game"].isin(
    ["Hades", "Tunic"])])
```

5.1 Unknown values in a condition

The rule A comparison with a missing value behaves differently in each representation.

- In R, a comparison with `NA` returns `NA`, and `filter` drops it because it is not `TRUE`.
- For a floating-point `NaN` in pandas, `NaN > 20` is `False` but `NaN != 20` is `True`.
- The nullable pandas `pd.NA` follows yet another set of rules.

Decide whether unknown values belong in the analysis, and write that decision into the condition with `is.na` or `isna`/`notna`.

Worked example `hours != 20` keeps seven records with `dplyr::filter` but all eight in pandas, because Dan's `NaN` passes. Base R bracket indexing with `hours > 20` returns four rows. The unknown condition requests an unknown row, filled with missing values, and does not recover an observed record. The explicit `known` table removes missing hours first and gives seven records in both languages.

R

```r
print(nrow(filter(plays, hours > 20)))   # 3
print(nrow(filter(plays, hours != 20)))  # 7
base_plays <- as.data.frame(plays)
print(nrow(
  base_plays[base_plays$hours > 20, ]))  # 4
known <- filter(plays, !is.na(hours),
                hours != 20)
print(nrow(known))  # 7, explicit NA policy
```

Python

```python
print(len(plays.loc[plays["hours"] > 20]))   # 3
print(len(plays.loc[plays["hours"] != 20]))  # 8
known = plays.loc[
    plays["hours"].notna()
    & (plays["hours"] != 20)]
print(len(known))  # 7, explicit NA policy
```

Differences to know

- `filter` drops a row whose condition is `NA`; a pandas mask treats `NaN` according to the comparison, so `!=` keeps it.
- Base R `df[cond, ]` with an `NA` condition adds an all-missing row; `filter` does not.
- Pandas keeps the original index labels after a filter; a tibble has no row labels to keep.

6 Choose variables and their names

What it is for Keep the columns a question needs, in a readable order, with suitable names.

The rule A selection can remove and reorder columns at once. A label range such as `player:hours` in dplyr or `"player":"hours"` with pandas `loc` includes both endpoints. An ordinary Python list or positional slice excludes its stop position. Selecting by type, with `where(is.numeric)` in dplyr or

`select_dtypes("number")` in pandas, relies on column types already checked during preparation. Renaming changes a label, not the values.

Worked example `player:hours` returns `player`, `game` and `hours`. The numeric selection returns `hours` and `achieved`.

R

```r
print(select(plays, player, game, hours))
print(select(plays, -achieved))
print(select(plays, player:hours))
print(select(plays, where(is.numeric)))
print(rename(plays, hrs = hours))
```

Python

```python
print(plays[["player", "game", "hours"]])
print(plays.drop(columns="achieved"))
print(plays.loc[:, "player":"hours"])
print(plays.select_dtypes("number"))
print(plays.rename(columns={"hours": "hrs"}))
```

7 Order records

What it is for Put records in an order that communicates something to the reader.

The rule `arrange` and `sort_values` sort by one or more keys. On these in-memory tables, missing values sort last in dplyr and in pandas with `na_position="last"`. A stable single-key sort keeps tied records in their previous relative order; pandas exposes this as `kind="stable"`, which applies to a single sort column. When the order of ties matters to a report, name additional ordering columns.

Worked example Sorting by descending hours starts with Ben–Hades at 45.5 hours and ends with Dan–Portal, whose hours are unknown.

R

```r
print(arrange(plays, desc(hours)))
print(arrange(plays, player, desc(achieved)))
```

Python

```python
print(plays.sort_values(
    "hours", ascending=False,
    kind="stable", na_position="last"))
print(plays.sort_values(
    ["player", "achieved"],
    ascending=[True, False]))
```

8 Derive a quantity with a valid denominator

What it is for Add a new column computed from existing ones.

The formula The rate is

$$\text{rate} = \frac{\text{achieved}}{\text{hours}} \quad \text{achievements per hour,}$$

computed only where hours are known and positive. The denominator column keeps positive hours and sets anything else to missing. The band is `"hi"` for a rate above 0.5 and `"lo"` otherwise. The bands are arbitrary teaching labels, not a measure of skill.

The rule `mutate` and `assign` evaluate new columns in order, so a later expression can use a column created earlier in the same call. In Python, `lambda d` names the table at that stage. Mutation can also replace or remove a column. Assigning the result to `rated` leaves `plays` unchanged.

Worked example Ana–Portal has $9/12.5 = 0.72$, band `hi`. Ben–Portal has $4/8 = 0.5$, band `lo`, because 0.5 is not above 0.5. Dan–Portal has no rate and no band. R's `if_else` propagates the unknown condition to `NA`; in Python the final `where` keeps the band missing so that an unknown rate is not labelled `lo`.

R

```r
rated <- mutate(plays,
  denominator = if_else(hours > 0, hours,
                        NA_real_),
  rate = achieved / denominator,
  band = if_else(rate > 0.5, "hi", "lo"))
print(select(rated, player, game, rate, band))
```

Python

```python
rated = plays.assign(
    denominator=lambda d: d.hours.where(
        d.hours > 0),
    rate=lambda d: d.achieved / d.denominator,
    band=lambda d: d.rate.gt(0.5)
        .map({True: "hi", False: "lo"})
        .where(d.rate.notna()))
print(rated[["player", "game", "rate", "band"]])
```

9 State how many values a summary uses

What it is for Collapse a table to a few numbers, and report the count each number rests on.

The rule A row count and a nonmissing-value count answer different questions. R's `mean` returns `NA` if any value is missing unless `na.rm=TRUE`. The pandas mean skips missing values by default; `skipna=False` makes it propagate. A sum that drops every missing value can be zero even when nothing was observed, so print the contributing count next to the result.

Worked example There are eight records and four players. Seven hours values are observed, summing to 141.5, so the mean is $141.5/7 = 20.214286$ hours per observed record. The achievement total is 76.

R

```r
print(mean(plays$hours))  # NA
overview <- summarise(plays,
  records = n(),
  players = n_distinct(player),
  observed_hours = sum(!is.na(hours)),
  mean_hours = mean(hours, na.rm = TRUE),
  achievements = sum(achieved))
print(overview)
# 8 records, 4 players, 7 observed,
# 20.214286 h, 76 achievements
```

Python

```python
print(plays["hours"].mean(skipna=False))  # nan
overview = pd.DataFrame([{
    "records": len(plays),
    "players": plays["player"].nunique(),
    "observed_hours": plays["hours"].count(),
    "mean_hours":
        plays["hours"].mean(skipna=True),
    "achievements": plays["achieved"].sum()}])
print(overview)
# 8 records, 4 players, 7 observed,
# 20.214286 h, 76 achievements
```

10 Groups and pipelines

What it is for Compute one summary per group, and chain operations so the code reads in the order the analysis runs.

The pipe In R, `x |> f()` passes `x` as the first argument of `f`, so `plays |> group_by(player)` groups the play records. The tidyverse `%>%` pipe does the same. In pandas, a method chain passes each result to the next method; parentheses let the chain span lines, and `pipe` passes a table to a separate function. The pattern is to choose groups, compute one summary per group, then drop the grouping before passing the result on. This is the split-apply-combine strategy [8].

Worked example Per player, the summary keeps the record count, the number of known hours values, their sum and the achievement count.

- Ana: 2 records, 2 known, 42.5 hours, 23 achievements.
- Ben: 2 records, 2 known, 53.5 hours, 29 achievements.
- Cara: 2 records, 2 known, 39.5 hours, 19 achievements.
- Dan: 2 records, 1 known, 6 hours, 5 achievements.

Dan's six hours come from Tunic alone, so they are a partial observed sum, not a player total. R's `.groups="drop"` returns an ungrouped result; pandas' `as_index=False` keeps `player` as a column.

R

```r
by_player <- plays |>
  group_by(player) |>
  summarise(records = n(),
    observed_hours = sum(!is.na(hours)),
    known_hours = sum(hours, na.rm = TRUE),
    achievements = sum(achieved),
    .groups = "drop")
print(by_player)
```

Python

```python
by_player = (plays
    .groupby("player", as_index=False)
    .agg(records=("game", "size"),
         observed_hours=("hours", "count"),
         known_hours=("hours", "sum"),
         achievements=("achieved", "sum")))
print(by_player)
```

10.1 A group summary versus a group-derived column

The rule `summarise` and `agg` collapse each group to one row. A grouped `mutate` in R, or `transform` in pandas, computes the group value and puts it back on every original row.

Worked example Each record's hours divided by its player's known hours keeps all eight records. Ana's Portal share is $12.5/42.5 = 0.294118$ and her Celeste share is $30/42.5 = 0.705882$. Dan's Tunic share is 1 and his Portal share stays missing. That 1 is a share of *known* hours, not evidence that Tunic is all of Dan's activity.

R

```r
shares <- plays |>
  group_by(player) |>
  mutate(share_known_hours =
    hours / sum(hours, na.rm = TRUE)) |>
  ungroup()
print(select(shares, player, game,
             share_known_hours))
```

Python

```python
shares = plays.assign(share_known_hours=(
    plays.hours
    / plays.groupby("player").hours
      .transform("sum")))
print(shares[["player", "game",
              "share_known_hours"]])
```

10.2 Order of the group results

The rule Grouping does not reorder the original rows. The order of groups in a summary is a separate choice. With character keys, dplyr's `group_by` and pandas' default `groupby` sort the keys. R's

per-operation `.by` and pandas' `sort=False` keep first appearance. Arrange the final table explicitly whenever order carries meaning.

Worked example Reversing the player order puts Dan first. With first-appearance grouping the summary rows come out as dan, cara, ben, ana.

R

```r
dan_first <- arrange(plays, desc(player))
print(summarise(dan_first,
  achievements = sum(achieved), .by = player))
# First appearance: dan, cara, ben, ana.
```

Python

```python
dan_first = plays.sort_values(
    "player", ascending=False)
print(dan_first.groupby("player", sort=False,
                        as_index=False)
      .agg(achievements=("achieved", "sum")))
# First appearance: dan, cara, ben, ana.
```

10.3 Empty categories

The rule A factor or categorical column can list levels that no row uses. R's `.drop=FALSE` and pandas' `observed=False` keep those levels as groups with a count of zero; the true settings drop them. Pandas 3.0 changed its default to observed categories only, so state the option.

Worked example Adding the level `eve` and keeping empty groups gives five rows, with Eve at zero records. Dropping empty groups gives four. The zero says Eve has no rows in this table. It does not show why, or that she played nothing elsewhere.

R

```r
levels_player <- c("ana", "ben", "cara",
                   "dan", "eve")
p <- mutate(plays,
  player = factor(player,
                  levels = levels_player))
empty_groups <- p |>
  group_by(player, .drop = FALSE) |>
  summarise(records = n(), .groups = "drop")
print(empty_groups)  # eve has 0 recorded rows
```

Python

```python
levels_player = ["ana", "ben", "cara", "dan",
                 "eve"]
p = plays.assign(player=pd.Categorical(
    plays.player, categories=levels_player))
empty_groups = (p
    .groupby("player", observed=False,
             as_index=False)
    .agg(records=("game", "size")))
print(empty_groups)  # eve has 0 recorded rows
```

10.4 A complete rate pipeline

What it is for Combine all six operations in one readable chain.

The steps Keep records with known positive hours, add the rate, group by player, average the record-level rates, sort the player summaries and select the output columns.

Worked example Ana's mean rate is

$$\frac{9/12.5 + 14/30}{2} = \frac{0.72 + 0.466667}{2} = 0.593333$$

achievements per hour. Her total achievements over total hours, $23/42.5 = 0.541176$, answers a different question, because it weights each record by its hours. Name the summary that matches the question. The four means in descending order are Ana 0.593333, Ben 0.524725, Dan 0.5 and Cara 0.478571. Dan contributes one eligible record; the excluded one still exists in `plays`.

R

```r
player_rates <- plays |>
  filter(!is.na(hours), hours > 0) |>
  mutate(rate = achieved / hours) |>
  group_by(player) |>
  summarise(records = n(),
            mean_rate = mean(rate),
            .groups = "drop") |>
  arrange(desc(mean_rate)) |>
  select(player, records, mean_rate)
print(player_rates)
```

Python

```python
player_rates = (plays
    .loc[plays.hours.notna()
         & (plays.hours > 0)]
    .assign(rate=lambda d: d.achieved / d.hours)
    .groupby("player", as_index=False)
    .agg(records=("game", "size"),
         mean_rate=("rate", "mean"))
    .sort_values("mean_rate", ascending=False)
    [["player", "records", "mean_rate"]])
print(player_rates)
```

Differences to know

- `.groups="drop"` in R and `as_index=False` in pandas both return a plain table with the key as a column.
- Grouped `mutate` in R corresponds to `groupby(…).transform` in pandas; `summarise` corresponds to `agg`.
- `group_by` sorts keys; `.by` keeps first appearance. Pandas sorts by default and keeps first appearance with `sort=False`.
- Empty levels are kept with `.drop=FALSE` in R and `observed=False` in pandas.

11 Combine tables by binding

What it is for Put two tables together when they describe the same kind of record (rows) or the same records (columns).

The rules `bind_rows` and pandas `concat` append rows, align columns by name and fill absent fields with missing values, subject to type compatibility. Base R's `rbind` rejects a different column set. Check column names, units and types before binding.

Adding columns is a different operation. `bind_cols` aligns rows by position and allows size-one recycling; pandas `concat(axis=1)` aligns index labels. Neither rule proves that paired rows describe the same entity. Check the identifiers first, and prefer a join when a key exists. Do not reset unrelated indexes just to make two tables line up.

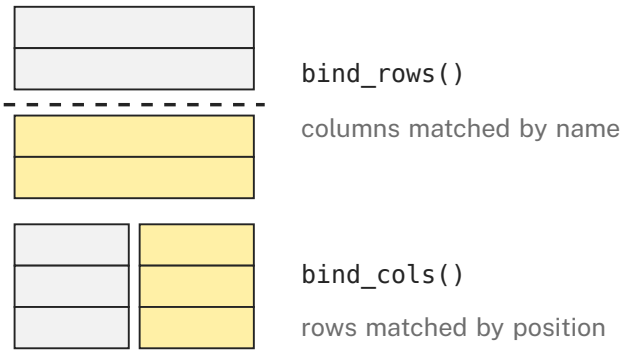


Figure 3. Two binding directions with different alignment requirements.

Worked example Appending a Tunic lookup row creates a five-row `extended`; `games` keeps four rows for the join examples. Binding `x = 1` with `y = 2` gives two rows and two columns, with a missing

value in each row. The column example checks that both `id` vectors are `a, b` in the same order before attaching `label`.

R

```r
extra <- tibble(game = "Tunic",
                genre = "puzzle", total = 42L)
extended <- bind_rows(games, extra)
print(extended)  # games itself still has 4 rows
a <- tibble(x = 1)
b <- tibble(y = 2)
print(bind_rows(a, b))  # missing cols become NA
```

Python

```python
extra = pd.DataFrame({"game": ["Tunic"],
    "genre": ["puzzle"], "total": [42]})
extended = pd.concat([games, extra],
                     ignore_index=True)
print(extended)  # games itself still has 4 rows
a = pd.DataFrame({"x": [1]})
b = pd.DataFrame({"y": [2]})
print(pd.concat([a, b], ignore_index=True))
```

R

```r
x <- tibble(id = c("a", "b"), value = c(2, 3))
y <- tibble(id = c("a", "b"),
            label = c("low", "high"))
stopifnot(identical(x$id, y$id))
print(bind_cols(x, select(y, label)))
```

Python

```python
x = pd.DataFrame({"id": ["a", "b"],
                  "value": [2, 3]})
y = pd.DataFrame({"id": ["a", "b"],
                  "label": ["low", "high"]})
assert x["id"].equals(y["id"])
print(pd.concat([x, y[["label"]]], axis=1))
```

12 Join on an explicit key

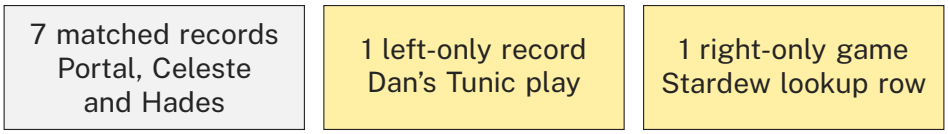
What it is for Attach lookup information to each record by matching a key, rather than by position.

The rule A join matches keys and returns every matching row combination. Here each game occurs once in `games` and several times in `plays`, so the relationship from plays to games is many-to-one. The four joins differ only in which unmatched rows they keep.

- Inner keeps matched combinations only.
- Left also keeps unmatched rows from the left table.
- Right also keeps unmatched rows from the right table.
- Full (outer) keeps unmatched rows from both.

Check before the merge that neither key column has missing values and that the lookup key is unique. `relationship="many-to-one"` in dplyr and `validate="many_to_one"` in pandas make the expectation executable.

Worked example Seven play records match a game, Tunic is left-only and Stardew is right-only. Inner gives 7 rows, left gives $7 + 1 = 8$, right gives $7 + 1 = 8$ and full gives $7 + 1 + 1 = 9$. The left join keeps all eight play records. The Stardew row in the right and full joins is an unmatched lookup entry, not a recorded play.



`inner_join` 7 matched records

`left_join` 7 + 1 = 8 records, including Tunic

`right_join` 7 + 1 = 8 rows, including Stardew

`full_join` 7 + 1 + 1 = 9 rows, including both

Figure 4. Exact row accounting for the plays and games tables.

R

```r
stopifnot(!anyNA(plays$game),
          !anyNA(games$game))
stopifnot(!anyDuplicated(games$game))
joined <- left_join(plays, games, by = "game",
  relationship = "many-to-one",
  na_matches = "never")
stopifnot(nrow(joined) == nrow(plays))
print(joined)
```

Python

```python
assert plays["game"].notna().all()
assert (games["game"].notna().all()
        and games["game"].is_unique)
joined = plays.merge(games, on="game",
                     how="left",
                     validate="many_to_one")
assert len(joined) == len(plays)
print(joined)
```

R

```r
print(nrow(joined))                   # 8
print(nrow(inner_join(plays, games,
                      by = "game")))  # 7
print(nrow(right_join(plays, games,
                      by = "game")))  # 8
print(nrow(full_join(plays, games,
                     by = "game")))   # 9
```

Python

```python
print(len(joined))                    # 8
print(len(plays.merge(games, on="game",
                      how="inner")))  # 7
print(len(plays.merge(games, on="game",
                      how="right")))  # 8
print(len(plays.merge(games, on="game",
                      how="outer")))  # 9
```

12.1 Defaults that change the result

The rule Join defaults can change row order, missing-key matching, suffixes and column types. The example rejects missing keys before the merge, and R also sets `na_matches="never"`, so an unknown identifier is never treated as a match. Table 1 lists the policies to state.

Worked example A duplicate on the right multiplies matches. Two Portal lookup rows would turn each of the three Portal plays into two rows, so the left join would have 11 rows instead of 8. The many-to-one check rejects that input. A missing integer lookup value can appear as floating-point `NaN` in a NumPy-backed pandas column; the nullable `Int64` type keeps integers with missing values.

The row count is one check. Also list which keys did not match, and keep lookup-only rows apart from observed records.

Table 1. Join policies to state when combining evidence.

Check	dplyr	pandas
Lookup cardinality	`relationship = "many-to-one"`	`validate="many_to_one"`
Missing keys	May match by default; `na_matches="never"` prevents this	May match by default; check or separate missing keys before merging
Output order	Left rows first, then unmatched right rows	Outer merge sorts keys; sort explicitly if order matters
Name clashes	Default suffixes `.x`, `.y`	Default suffixes _x, _y

Differences to know

- `left_join(…, relationship="many-to-one")` and `merge(…, validate="many_to_one")` both stop on a duplicated lookup key.
- dplyr's full join is pandas' `how="outer"`.
- `na_matches="never"` has no pandas argument; the example asserts that neither key column has missing values instead.
- `bind_cols` aligns by position; `concat(axis=1)` aligns by index label.

13 Graphics systems and output devices

What it is for Know where a drawing command sends its output, so a script produces the same file every time.

The data The graphics sections use the Iris measurements: 150 flowers, sepal and petal measurements in centimetres, three species [1, 4]. Changing data here keeps the plotting interface apart from the join example. The figure for the game data comes in Section 15.

The R graphics path R has two graphics systems. Base graphics serves `plot` and `hist`; packages such as `maps` can also draw through it. Grid underlies `ggplot2` and `lattice`. Both send drawings through the graphics engine to a graphics device. The `grDevices` package provides the device functions; other device packages exist but none is needed here. Murrell describes this structure in detail [3].

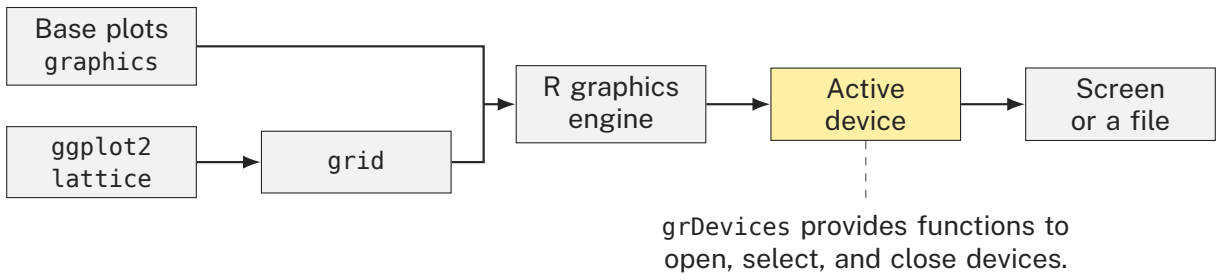


Figure 5. R graphics systems, the graphics engine, and output devices. After Murrell [3].

Table 2. Responsibilities in the R graphics path.

Component	Role
`graphics`	Base plotting system, such as `plot` and `hist`
`grid`	Drawing system used by `ggplot2` and `lattice`
Graphics engine	Connects drawing systems to graphics devices
`grDevices`	Device-management functions, including `png`, `pdf`, and `dev.off`
Device	Sends the drawing to a display or a chosen file format

13.1 Choose the destination explicitly

The rule R draws on the active device. Several devices can be open, but only one is active. An interactive session often draws on screen, while a script may use a different default, so name the file device.

1. Open the device with a pixel size and resolution for PNG, or a page size for PDF.
2. Add the plotting calls.
3. Close it with `dev.off` so the file is completed. An unclosed device can leave incomplete output, depending on the device and how the session ends; it does not always leave an empty file.

Matplotlib keeps explicit Figure and Axes objects. Draw on the chosen axes, save the figure, and close it, which matters when a script makes several figures.

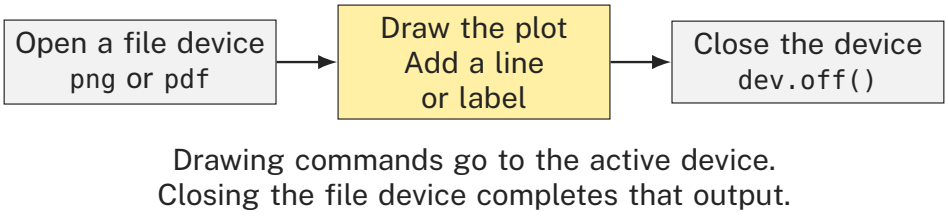


Figure 6. An explicit file-device lifecycle.

13.2 An Iris scatter plot with a reference line

Worked example Draw sepal width against sepal length, then add a horizontal line at the overall mean width, 3.057333 cm. The line is one marginal summary, not a regression line and not an explanation of the association. Both scripts save a 1200 by 800 pixel PNG with hollow points, a gold mean line and centimetre labels. R's `abline` draws on the current plot; Python's `ax.axhline` names the axes.

R

```r
d <- as_tibble(iris)
print(nrow(d))                  # 150 flowers
print(mean(d$Sepal.Width))      # 3.057333 cm
```

Python

```python
raw = load_iris(as_frame=True)
d = raw.data.copy()
d.columns = ["Sepal.Length", "Sepal.Width",
             "Petal.Length", "Petal.Width"]
d["Species"] = pd.Categorical.from_codes(
    raw.target, raw.target_names)
print(len(d), d["Sepal.Width"].mean())
# 150, 3.057333 cm
```

R

```r
png("class07-iris-base.png", width = 1200,
    height = 800, res = 150)
plot(d$Sepal.Length, d$Sepal.Width, pch = 1,
     xlab = "Sepal length (cm)",
     ylab = "Sepal width (cm)")
abline(h = mean(d$Sepal.Width),
       col = "#A88800", lwd = 2)
dev.off()
```

Python

```python
fig, ax = plt.subplots(figsize=(8, 16 / 3))
ax.scatter(d["Sepal.Length"], d["Sepal.Width"],
           facecolors="none",
           edgecolors="black")
ax.axhline(d["Sepal.Width"].mean(),
           color="#A88800", linewidth=2)
ax.set(xlabel="Sepal length (cm)",
       ylabel="Sepal width (cm)")
fig.savefig("class07-iris-base.png", dpi=150)
plt.close(fig)
```

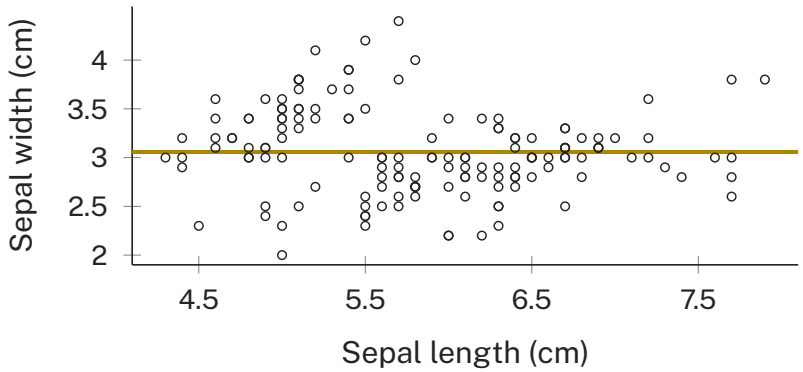


Figure 7. All 150 Iris measurements. Lengths and widths are recorded to 0.1 cm, so the 150 flowers occupy 117 distinct positions and overlapping points hide 33 of them. The gold reference line marks the overall mean sepal width.

14 The grammar of graphics

What it is for Describe any statistical graphic as a set of separate choices, so each choice can be changed on its own [7, 11, 12].

The six components Table 3 applies each one to Iris.

- **Data** supplies the values.
- **Aesthetic mappings** link variables to visible properties.
- A **layer** combines a geometric object (the marks) with a statistical transformation, such as identity for raw points or counts for histogram bins.
- **Scales** convert data values to display values. A scale need not be one-to-one; several values can share one binned colour.
- The **coordinate system** sets the panel geometry.
- **Facets** split the data into panels.

Many components have defaults, but data and mappings alone draw nothing; a layer is required. Changing the geometric object can also require new mappings or a new calculation, because a histogram and a scatter plot ask different questions.

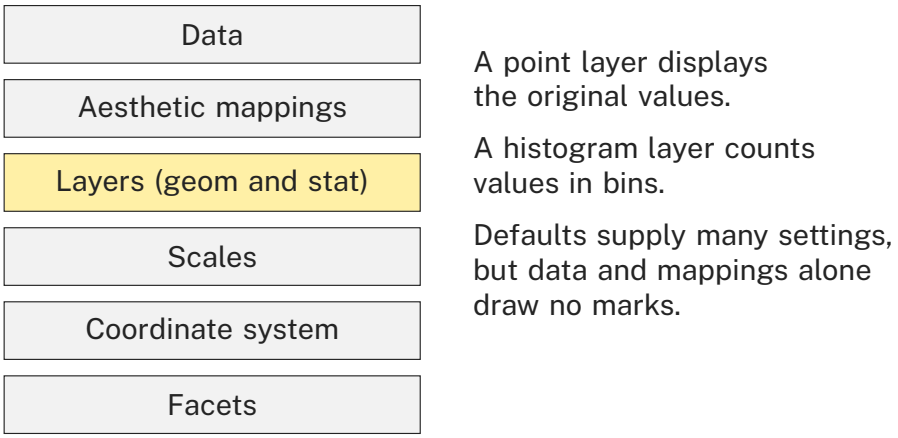


Figure 8. The six components of a layered statistical graphic.

Table 3. The six components applied to the Iris example. The layer takes two rows, its geometric object and its statistical transformation.

Element	Meaning	Iris example
Data	Values to display	Flower measurements
Aesthetic mappings	Variables linked to visible properties	Length to x, width to y
Geometric objects	Marks that display values	One point per flower
Statistical transformations	Calculations used by a layer	Identity for points; counts for histogram bins
Scales	Convert values into positions, colours, or other properties	Numeric axes and species colours
Coordinate system	Geometry of the plotting panel	Cartesian x and y
Faceting	Separate panels for subsets	One panel per species

14.1 Panels, layers, and explicit styling

Worked example Map length to x and width to y, add a point layer, add the overall mean width (3.057333 cm) to every panel, and facet by species. The facet headings identify the species without relying on colour. R sets `theme_minimal` and Python sets seaborn's white-grid style; these are choices in the code, not package defaults.

Stored specification versus immediate drawing A ggplot object stores a plot specification that can be extended and saved. Base R draws immediately; display lists can be recorded, but an ordinary `plot` call is not a stored specification. In Python, Matplotlib offers Figure and Axes objects, and seaborn offers chart-family functions.

R

```r
colours <- c("#1F5C8B", "#2E7D5B", "#6A3D9A")
p_iris <- ggplot(d,
  aes(Sepal.Length, Sepal.Width,
      colour = Species)) +
  geom_point() +
  geom_hline(yintercept = mean(d$Sepal.Width),
             colour = "#A88800") +
  facet_wrap(~ Species) +
  scale_colour_manual(values = colours) +
  labs(x = "Sepal length (cm)",
       y = "Sepal width (cm)") +
  theme_minimal()
ggsave("class07-iris-facets.pdf", p_iris,
       width = 9, height = 3.5)
```

Python

```python
colours = ["#1F5C8B", "#2E7D5B", "#6A3D9A"]
sns.set_theme(style="whitegrid")
g = sns.relplot(
    data=d, x="Sepal.Length", y="Sepal.Width",
    hue="Species", col="Species",
    kind="scatter", palette=colours,
    height=3.5, aspect=6 / 7)
g.refine(y=d["Sepal.Width"].mean(),
         color="#A88800")
g.set_axis_labels("Sepal length (cm)",
                  "Sepal width (cm)")
g.savefig("class07-iris-facets.pdf")
plt.close(g.figure)
```

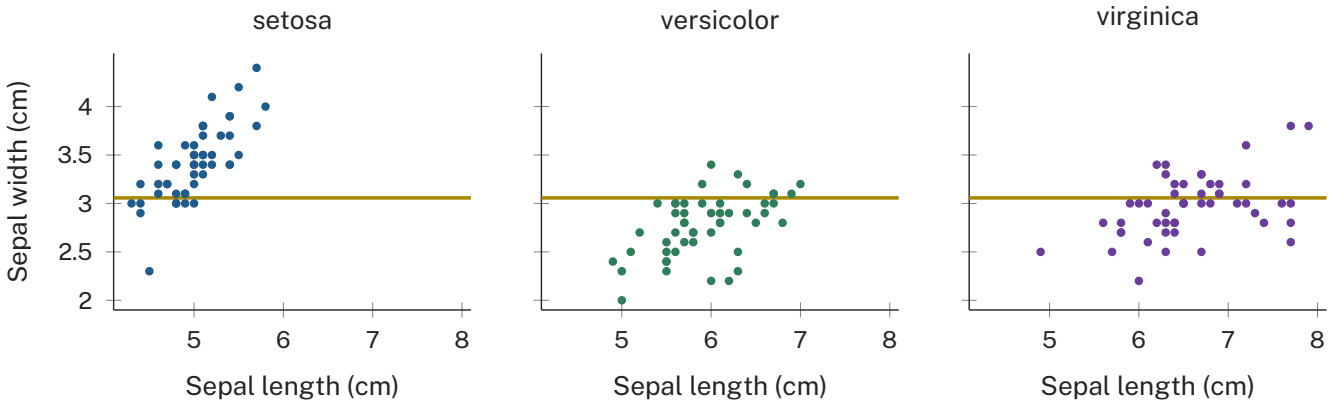


Figure 9. The faceted Iris plot. The panels share both axes. Of 50 flowers per species, 42 setosa, 8 versicolor and 17 virginica lie above the gold mean width.

Differences to know

- R draws on the active device and needs `dev.off`; Matplotlib saves a named Figure with `savefig` and closes it with `plt.close`.
- `abline` targets the current plot; `ax.axhline` and seaborn's `refline` target named axes.
- `facet_wrap(~ Species)` corresponds to `col="Species"` in `sns.relplot`.

15 From a checked join to an explained figure

What it is for Carry one question through eligibility, a grouped summary and a saved figure, keeping a record of every excluded row.

The question What is the mean achievement completion percentage per genre, among play records with known hours greater than five and a known positive achievement total? The threshold of five is an example eligibility rule, not a recommendation.

The rule Decide eligibility before grouping, and keep the excluded rows in their own table. Without an explicit rule, R keeps a missing-genre group while pandas drops it by default, so the two languages print different tables.

Worked example, eligibility Of the eight joined records, six are eligible. Dan–Portal is excluded because its hours are unknown, and Dan–Tunic because its lookup values (genre and total) are missing.

R

```r
checked <- mutate(joined,
  eligible = !is.na(hours) & hours > 5 &
    !is.na(total) & total > 0 & !is.na(genre))
excluded <- filter(checked, !eligible)
eligible <- filter(checked, eligible)
print(select(excluded, player, game, hours,
             genre, total))
print(c(input = nrow(plays),
        eligible = nrow(eligible),
        excluded = nrow(excluded)))  # 8, 6, 2
```

Python

```python
checked = joined.assign(eligible=
    joined.hours.notna() & (joined.hours > 5) &
    joined.total.notna() & (joined.total > 0) &
    joined.genre.notna())
excluded = checked.loc[~checked.eligible].copy()
eligible = checked.loc[checked.eligible].copy()
print(excluded[["player", "game", "hours",
                "genre", "total"]])
print(len(plays), len(eligible),
      len(excluded))  # 8, 6, 2
```

The formula For each eligible record,

$$\text{pct} = 100 \times \frac{\text{achieved}}{\text{total}},$$

then average the percentages within each genre, giving every record equal weight.

Worked example, genre summary

- Puzzle: $(60 + 26.666667)/2 = 43.333333\%$, from 9/15 and 4/15.
- Platformer: $(46.666667 + 36.666667)/2 = 41.666667\%$, from 14/30 and 11/30.
- Roguelike: $(51.020408 + 16.326531)/2 = 33.673469\%$, from 25/49 and 8/49.

Each genre has two eligible records, with known hours of 20.5, 52 and 63 respectively.

R

```r
genre_summary <- eligible |>
  mutate(pct = 100 * achieved / total) |>
  group_by(genre) |>
  summarise(records = n(),
            known_hours = sum(hours),
            pct = mean(pct),
            .groups = "drop") |>
  arrange(desc(pct))
print(genre_summary)
```

Python

```python
genre_summary = (eligible
    .assign(pct=lambda d:
            100 * d.achieved / d.total)
    .groupby("genre", as_index=False)
    .agg(records=("player", "size"),
         known_hours=("hours", "sum"),
         pct=("pct", "mean"))
    .sort_values("pct", ascending=False))
print(genre_summary)
```

Drawing the summary Print the table before drawing it. The table already holds one value per genre, so draw those values directly with `geom_col` or Matplotlib bars. The zero baseline makes bar lengths comparable. Save the chart, the summary CSV and the exclusion CSV together.

R

```r
p_genre <- ggplot(genre_summary,
  aes(reorder(genre, -pct), pct)) +
  geom_col(fill = "#1F5C8B",
           colour = "#242424") +
  geom_text(aes(label = round(pct, 1)),
            vjust = -0.4) +
  scale_y_continuous(limits = c(0, 50)) +
  labs(x = "Genre",
       y = "Mean achievement completion (%)") +
  theme_minimal()
ggsave("class07-genre-summary.pdf", p_genre,
       width = 7, height = 4)
```

Python

```python
fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.bar(genre_summary.genre,
              genre_summary.pct,
              color="#1F5C8B",
              edgecolor="#242424")
ax.bar_label(bars, fmt="%.1f")
ax.set(xlabel="Genre",
       ylabel="Mean achievement completion (%)",
       ylim=(0, 50))
fig.tight_layout()
fig.savefig("class07-genre-summary.pdf")
plt.close(fig)
```

Then save the summary table and the exclusion record beside the figure.

R

```r
write.csv(genre_summary,
          "class07-genre-summary.csv",
          row.names = FALSE)
write.csv(excluded, "class07-excluded.csv",
          row.names = FALSE)
```

Python

```python
genre_summary.to_csv(
    "class07-genre-summary.csv", index=False)
excluded.to_csv("class07-excluded.csv",
                index=False)
```

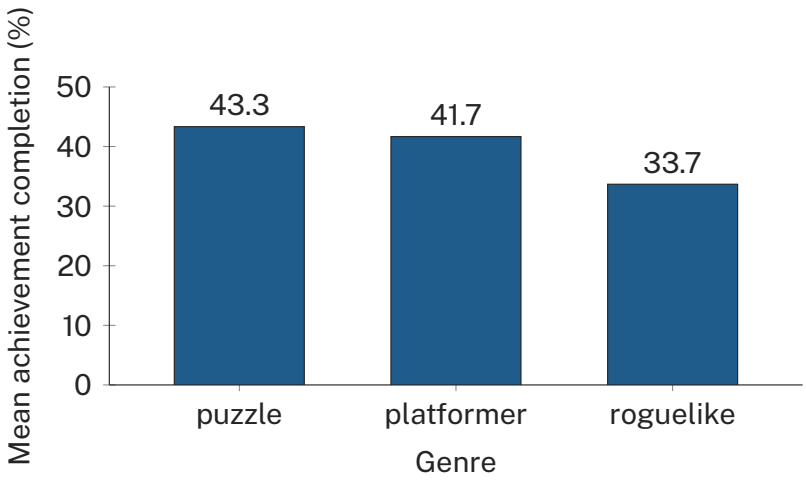


Figure 10. Mean completion across six eligible play records, two in each genre. Puzzle leads platformer by 1.7 percentage points; roguelike trails it by 8.0.

16 Summary

- Say what one row represents, and how many values each summary uses.
- Write the missing-value policy into the code with `is.na` or `notna`; never inherit it from a library default.
- Grouping does not collapse records. A summary collapses them; a grouped mutate or transform keeps them.
- An absent category does not prove zero activity, and an unknown value must not fall into a low band by accident.
- A mean of record rates and a rate from totals answer different questions.

- Keep the original tables, and keep a table of excluded records.
- Validate lookup keys before a join, and tell unmatched rows apart from duplicated matches.
- Choose the output device explicitly, and save a labelled figure with its numerical summary and the code that produced both.

References

[1] R. A. Fisher. “The use of multiple measurements in taxonomic problems”. In: *Annals of Eugenics* 7.2 (1936), pp. 179–188.
[2] K. Müller and H. Wickham. *tibble: Simple Data Frames*. Version 3.3.1. Package documentation and the `tibble` vignette. CRAN. 2026.
[3] P. Murrell. *R Graphics*. 3rd ed. Chapman and Hall/CRC, 2018.
[4] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. Vienna, Austria, 2026.
[5] The pandas development team. *pandas User Guide*. Version 3.0.6. “Merge, join, concatenate and compare”, “Group by: split-apply-combine”, “Working with missing data”. 2026.
[6] M. L. Waskom. “seaborn: statistical data visualization”. In: *Journal of Open Source Software* 6.60 (2021), p. 3021.
[7] H. Wickham. “A layered grammar of graphics”. In: *Journal of Computational and Graphical Statistics* 19.1 (2010), pp. 3–28.
[8] H. Wickham. “The split-apply-combine strategy for data analysis”. In: *Journal of Statistical Software* 40.1 (2011), pp. 1–29.
[9] H. Wickham. “Tidy data”. In: *Journal of Statistical Software* 59.10 (2014), pp. 1–23.
[10] H. Wickham, R. François, L. Henry, K. Müller, and D. Vaughan. *dplyr: A Grammar of Data Manipulation*. Version 1.2.1. CRAN. 2026.
[11] H. Wickham, D. Navarro, and T. L. Pedersen. *ggplot2: Elegant Graphics for Data Analysis*. 3rd ed. Online edition, work in progress, `https://ggplot2-book.org/`.
[12] L. Wilkinson. *The Grammar of Graphics*. 2nd ed. Springer, 2005.